L05 – Multilayer Perceptron (MLP)

Foundations Lecture Series

Course Instructor

Microgrid 3-phase Security AI Project

Foundations Lecture 5

Keywords: layers, activations, backprop, loss, train/val/test, PyTorch

Lecture roadmap

1. From linear models to MLPs
2. Architecture: layers, activations, output heads
3. Training: loss, backprop, gradient descent
4. The discipline: train/val/test, regularization, class imbalance
5. PyTorch minimal example
6. Common pitfalls

Prerequisite

Basic linear algebra (matrix-vector products, dot products) and basic calculus (partial derivatives, chain rule). Python familiarity.

Why linear is not enough

TODO: *Show a simple example where linear classification fails (XOR or moons). State that an MLP is just a sequence of linear maps composed with nonlinearities. Visualize how stacking transforms separates the data.*

The MLP equation

TODO: *Write the standard form: $h_{l+1} = \sigma(W_l h_l + b_l)$. Define each symbol. Show the data shape flow from input through hidden to output. Note the connection to logistic regression as the 0-hidden-layer case.*

Activation functions

TODO: *Plot and explain: sigmoid, tanh, ReLU, LeakyReLU, GELU. State practical defaults: ReLU for hidden, sigmoid/softmax for output. Note vanishing gradient problem of sigmoid in deep nets.*

Output heads for different tasks

TODO: *Binary classification: sigmoid + BCE. Multi-class: softmax + cross-entropy. Regression: linear + MSE. Multi-task: multiple heads. Forward-reference to PI-MLP design in Week 6 of the project.*

Choosing depth and width

TODO: *Rules of thumb: start small (1–2 hidden layers, 32–128 units), increase if underfitting. Connect to bias-variance: too small underfits, too large overfits on small data. Reference the project's 77-sample regime as "small data".*

The training loop in one slide

TODO: *Pseudocode: for each epoch, for each batch, (1) forward pass, (2) compute loss, (3) backward pass, (4) update parameters. Note the role of the optimizer.*

Backpropagation in one slide

TODO: *Chain rule applied layer-by-layer to compute $\partial \mathcal{L}/\partial W_l$ efficiently. Note that frameworks like PyTorch compute this automatically via autograd. No need to derive gradients by hand.*

Optimizers: SGD, momentum, Adam

TODO: *Short tour: vanilla SGD, SGD with momentum, Adam. Default for this course: Adam with lr=1e-3. Mention learning-rate schedulers briefly.*

Train / validation / test splits

TODO: *Define each, with the rule that test is touched once at the end. Validation is for hyperparameter tuning. Forward-reference: the project uses group CV (by scenario or contingency), not random splits.*

Regularization: why and how

TODO: *L2 weight decay, dropout, early stopping. Connect to overfitting in the project's small-sample regime. Show a generic train/val loss curve with the overfitting inflection point.*

Class imbalance

TODO: *The project's labels are 47 unsafe vs 30 safe. Discuss weighted BCE, oversampling, threshold tuning. Reinforce: in safety screening, the right metric is recall/FNR, not accuracy.*

Minimal MLP in PyTorch

TODO: *Code block: define a 2-hidden-layer MLP with torch.nn.Sequential, instantiate optimizer (Adam), define loss (BCE), training loop. Aim for* ≤ 25 *lines of code total.*

Inspecting a trained model

TODO: *Show how to: get predictions, compute confusion matrix, plot PR curve, compute recall at a chosen threshold. Frame these as the metrics that matter in the project.*

Pitfalls in MLP training

TODO: *Enumerate: not normalizing inputs; using the wrong activation on the output head; data leakage between train and test; tuning on the test set; declaring victory based on accuracy when the data is imbalanced; forgetting to seed the RNG.*

Homework

TODO: *Suggest: build a 2-hidden-layer MLP on the moons dataset, achieve 95%+ test accuracy, plot the decision boundary, then re-train with severe class imbalance and explain why the boundary moves.*

Exit checklist

TODO: *Items: write the MLP forward equation, list 3 activation functions and when to use each, explain backprop in 2 sentences, code a training loop in PyTorch, name 3 regularizers.*

References

TODO: *Goodfellow et al. Deep Learning chapter 6, Bishop Pattern Recognition and Machine Learning chapter 5, PyTorch tutorials (`nn.Module` and the 60-min blitz).*